Exercises Java Libraries

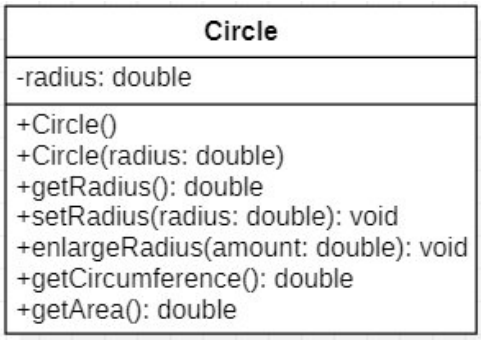
For all exercises:

- Use the start folders on Canvas
- Create the class(es) in the package *model* according to the given UML Diagram. No more and no less. **You are not allowed to create additional or other attributes or methods than the one you find in the UML Diagram**…
- After you have written these class(es), run the tests
- If all tests are successful, create objects of the class in the main method of your application and experiment with the different methods. Don't forget to import the class you have created…

Exercise 1

Create a class **Circle**

- the method *enlargeRadius* causes the radius to be increased by the *amount* parameter
- the method *getCircumference* returns the circumference ($2\pi r$)
- the *getArea* method returns the area (πr^2)



Search the Java API documentation of the class Math for constants/methods to solve this exercise.

Use the unit tests to check your class.

Now create an object in the **ExerciseCircleApplication** and print the next output:

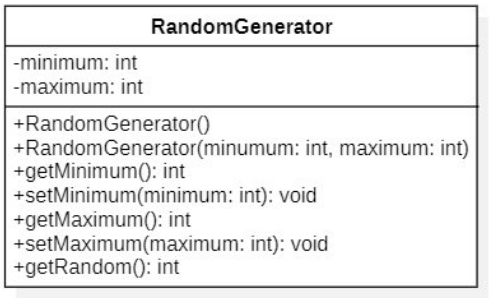
```
The circle with radius: 4.3 has
area = 58.09
circumference = 27.02
```

Tip: you can format the output using the static *format* method of the class String:

```
System.out.println(String.format("area = %.2f", area));
```

Exercise 2

Create a class **RandomGenerator**



- in the no-arg *constructor:* minimum=1 and maximum=5

- the *getRandom*() method returns a random number (integer) between a minimum value inclusive and a maximum value exclusive.
  To work out this method you use the class ***Random*** **(package java.util)**, which has interesting methods for generating random integers or double decimals. Search in the Java API:
  - which *constructor* you can use to create an object of this class
  - then check which *instance methods* this class has and find the nextInt(int bound) method. Find out how to use it correctly in this exercise.

Use the unit tests to check your class.

Try the getRandom() method by creating a RandomGenerator object in the ExerciseRandomGeneratorApplication to generate 10 random numbers between -50 and 50 (-50 and +50 inclusive).
The output can look like this, for example:

```
Random numbers between -50 and 50
-29 37 -15 35 -7 -22 34 11 -34 -49
```

Exercise 3

Create a class **DateAnalyzer**

DateAnalyzer
-date: LocalDate
+DateAnalyzer(date: String) +getNumberedDayOfTheYear(): int +getNumberedDayOfTheMonth(): int +getDayOfWeek(): DayOfWeek +getMonth(): Month +get100DaysOld(): String +get10MonthsOld(): String +getDate(): String

Pay attention to the constructor and the getter of Date! They are not the standard generated constructor and getter... The constructor expects a date in the form of a String e.g. "7/1/2002" (format = d/M/yyyy) and the getter returns a String showing a date in the form of a String e.g. "7-1-2002" (format = d-M-yyyy). The other methods with a return type String will return also a date in the form of a String with format = d-M-yyyy.

Take a look at the screenshot below to see how the methods work.

Use the unit tests to check your class.

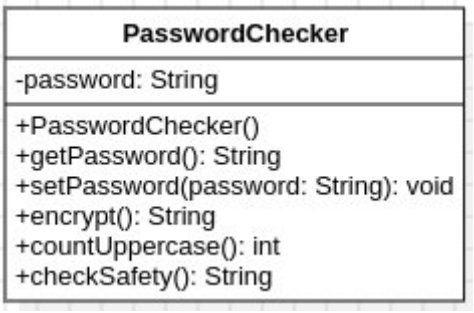
In the **ExerciseDateAnalyzerApplication** you generate a DateAnalyzer object by entering your date of birth in the format d/M/yyyy as a parameter.

You then print this information:

```
You were born on: 15-6-2001
Day of the month: 15
Day of year: 166
Weekday: FRIDAY
Month: JUNE
On that day you were 100 days old: 23-9-2001
On that day you were 10 months old: 15-4-2002
```

Exercise 4

Create a class **PasswordChecker**



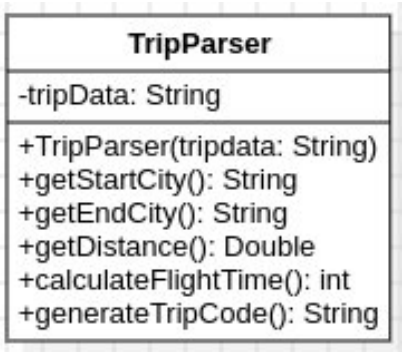
- The method **encrypt** returns a String in which the password is converted to "Leet Speak" (1337). This is a substitution cipher where:
    - 'a' or 'A' becomes '@'
    - 'e' or 'E' becomes '3'
    - 'i' or 'I' becomes '1'
    - 'o' or 'O' becomes '0'
    - The final step of this encryption takes the first half of the password (rounded down) and puts it at the end instead of the beginning.
- The method **checkSafety** returns "Safe" if the password is at least 8 characters long and contains at least one digit and one special character. Otherwise, it returns "Unsafe".
- The method **countUppercase** counts the number of uppercase letters in the password.

Use the unit tests to check your class.

In your ExercisePasswordCheckerApplication, ask the user for a password and print the following output for example for “Password123”:

```
The password is: Password123
Encrypted password: 0rd123P@ssw
Number of uppercase letters: 1
Password safety: Safe
```

Exercise 5



Create a class **TripParser**

- The **constructor** accepts a String containing trip data in a specific format: "StartCity-EndCity-Distance".
    - Example: "Paris-Berlin-878.5"
- The methods **getStartCity** and getEndCity return the names of the cities.
- The method **getDistance** extracts the numeric part from the string and returns it as a double.
- The method **calculateFlightTime** returns the estimated flight time in hours.
    - Assume an average speed of 900 km/h.
- The method **generateTripCode** creates a random unique code for the trip.
    - It combines the first 2 letters of the start city (*uppercase*).
    - It adds a random integer between 100 and 999.
    - It adds the last letter of the end city (*uppercase*).

Search the Java API for usefull methods to implement in this class.

Use the unit tests to check your class

In the ExerciseTripParserApplication ask the user to input the raw data string in the correct format. Afterwards, show the user all relevant data:

```
Please enter trip data (Format: Start-End-Distance):
Example: Paris-Berlin-878.5
Brussels-Hong Kong-9366.59

--- Trip Analysis ---
From: Brussels
To: Hong Kong
Distance: 9366.59 km
Estimated Flight Time: 11 hours
Trip Code: BR835G
```